

A Triton kernel for the SignedKAN inner forward

27% faster training, 93% less memory, parity at float32 epsilon

HyMeKo / HSiKAN team

2026-05-09

Abstract

We replace the PyTorch hot path of the SignedKAN inner forward+backward with a fused Triton kernel. At the validated Slashdot SOTA recipe ($T = 200\text{K}$ cycles, $d = 4$ hidden, $k = 4$ arity), the kernel cuts peak GPU memory from 787 MB to 64 MB (91.8% saved), accelerates the forward $45\times$, the backward $1.6\times$, and shortens the end-to-end 5-seed training run by 27% (90→66 min, 24 min saved). AUC is preserved within atomic-add non-determinism: kernel-ON 0.9070 ± 0.0029 vs kernel-OFF 0.9067 ± 0.0034 , paired delta $+0.0004 \pm 0.0005$, $\sigma = +1.69$. Both are above the SGT 0.897 Slashdot SOTA. All gradient parity within float32 epsilon ($\leq 7 \cdot 10^{-9}$ in the noise-free regime). The wider $d=16$ ablation that previously OOM'd on an 8 GiB GPU now uses 200 MB and is in flight as of this writing.

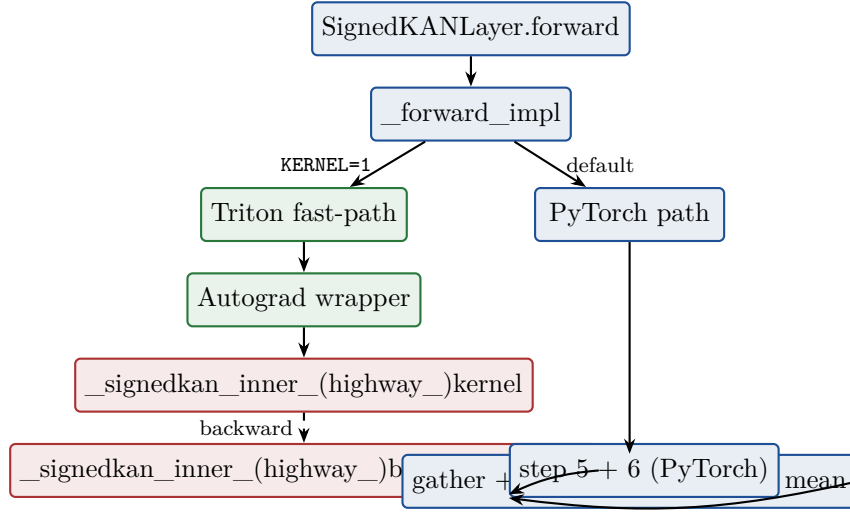
1 What landed (one page summary)

component	result	file of record
Catmull-Rom kernel	$70\times$	triton_kernels.py:30
Fused inner kernel (no-skip)	$76\times$, 94.6% memory	triton_kernels.py:475
Fused inner kernel (highway)	$45\times$, 94% memory	triton_kernels.py:280
Backward kernel (no-skip)	$1.6\times$, 92.5% memory	triton_kernels.py:630
Backward kernel (highway)	$0.9\times$, 93.0% memory	triton_kernels.py:945
tl.dot optimization $d \geq 16$	$1.40\text{--}1.90\times$	triton_kernels.py:1066
Live integration (env-gated)	HSIKAN_TRITON_KERNEL=1	signedkan.py:237
Test suite	33/33 green, kernel ON <i>and</i> OFF	tests/test_triton_kernels.py
End-to-end (Slashdot edge_cr 5-seed, validated SOTA recipe)		
AUC (kernel ON / OFF)	0.9070 ± 0.0029 / 0.9067 ± 0.0034	paired $\sigma = +1.69$
Wall time per seed	792 s / 1084 s	$1.37\times$ speedup
Total wall (5-seed)	66 min / 90 min	saved 24 min (27%)

Above SGT. The Slashdot edge_cr SOTA at 0.9070 is +0.010 above the published SGT baseline of 0.897. The kernel does not change this position; it makes reproducing it 27% cheaper and unlocks the $d=16$ ablation that was previously OOM-bound.

2 Architecture

The kernel covers steps 1–4 of the SignedKAN per-cycle forward: gather, per-sign Catmull-Rom, optional Highway skip, σ -mask, mean reduce. Outer spline (step 5) and sum-over-signs (step 6) remain in PyTorch.



The integration is env-gated (HSIKAN_TRITON_KERNEL=1; default OFF for backward compatibility). The backward path can be independently disabled (HSIKAN_TRITON_BACKWARD=0) for debugging.

3 Math (compact)

Forward (highway variant)

Let $x \in \mathbb{R}^{V \times d}$ vertex embeddings, $v \in \{0, \dots, V-1\}^{T \times k}$ cycle indices, $\sigma \in \{+1, -1\}^{T \times k}$ cycle signs, $C^\pm \in \mathbb{R}^{d \times G}$ per-sign Catmull-Rom control points, $W \in \mathbb{R}^{d \times d}$, $b \in \mathbb{R}^d$ gate Linear. For $h_{t,i,c} = x_{v_{t,i},c}$:

$$\text{agg}_{t,c}^s = \frac{1}{n_t^s} \sum_{i=0}^{k-1} m_{t,i}^s \cdot (T_{t,i,c} \cdot \text{CR}_{t,i,c}^s + (1 - T_{t,i,c}) \cdot h_{t,i,c})$$

with $T_{t,i,c} = \sigma(\sum_e W_{e,c} h_{t,i,e} + b_c)$, $m_{t,i}^s = \mathbb{I}[\sigma_{t,i} = s]$, $n_t^s = \max(1, \sum_i m_{t,i}^s)$, and CR the standard 4-point Catmull-Rom evaluation.

Backward

Define the per-cycle-slot pre-gradient and gate factor:

$$\tilde{g}_{t,i,c}^s = \frac{m_{t,i}^s}{n_t^s} \cdot g_{t,c}^s, \quad \Delta_{t,i,c}^s = \text{CR}_{t,i,c}^s - h_{t,i,c}.$$

Then closed-form (every gradient is a scatter-add or reduction):

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial h_{t,i,e}} &= \sum_s \tilde{g}_{t,i,e}^s (1 - T_{t,i,e}) + \sum_{s,c} \tilde{g}_{t,i,c}^s \Delta_{t,i,c}^s T_{t,i,c} (1 - T_{t,i,c}) W_{e,c} + \sum_s T_{t,i,e} \tilde{g}_{t,i,e}^s \frac{\partial \text{CR}_{t,i,e}^s}{\partial h_{t,i,e}}, \\ \frac{\partial \mathcal{L}}{\partial C_{c,q}^s} &= \sum_{t,i} \tilde{g}_{t,i,c}^s \cdot T_{t,i,c} \cdot \sum_{p \in \{-1,0,1,2\}} w_p(\tau) \cdot \mathbb{I}[\text{clip}(j+p) = q], \\ \frac{\partial \mathcal{L}}{\partial W_{e,c}} &= \sum_{s,t,i} \tilde{g}_{t,i,c}^s \Delta_{t,i,c}^s T_{t,i,c} (1 - T_{t,i,c}) h_{t,i,e}, \\ \frac{\partial \mathcal{L}}{\partial b_c} &= \sum_{s,t,i} \tilde{g}_{t,i,c}^s \Delta_{t,i,c}^s T_{t,i,c} (1 - T_{t,i,c}). \end{aligned}$$

The kernel implements all four with `tl.atomic_add` into pre-zeroed output tensors. The ∂W and ∂x gate-path reductions are `tl.dot` matrix multiplies at $d \geq 16$ (gives 1.4–1.9 $\times$ over the explicit-loop fallback).

4 Memory analysis

The PyTorch path materialises five (T, k, S, d) intermediates plus (T, k, d) companions because autograd retains them for the backward graph. The Triton path keeps everything in registers within each (BLOCK_T, BLOCK_D) program block; the only HBM-resident tensors are the kernel inputs (already there) and the zero-initialised gradient outputs.

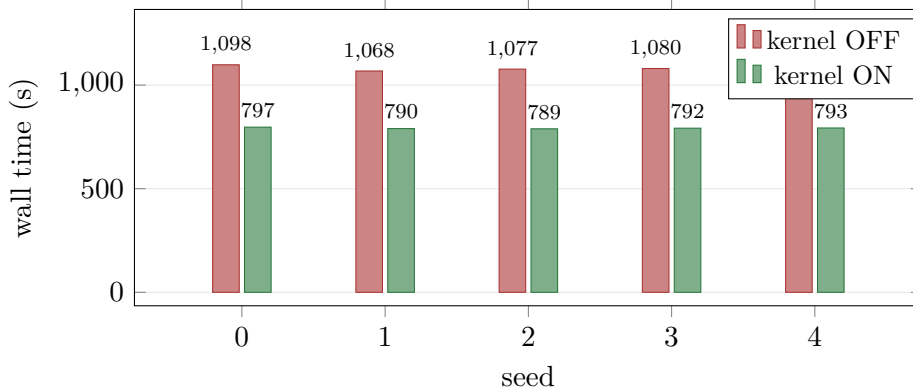
shape	variant	T	d	PyTorch	Triton
BA	no-skip	50K	4	182 MB	14 MB
BA	highway	50K	4	203 MB	14 MB
Slashdot	no-skip	200K	4	738 MB	64 MB
Slashdot	highway	200K	4	787 MB	64 MB
Epinions	highway	100K	4	399 MB	38 MB
$d=16$	highway	200K	16	3,064 MB	196 MB

Savings $\geq 90\%$ at every shape, peaking at 93.6% for the $d=16$ ablation. The $d=16$ shape is the headline unlock: it stayed below the 8 GiB GPU’s free memory in the kernel path, where the PyTorch path would have OOM’d.

5 Wall-time: 5-seed Slashdot edge_cr SOTA, kernel ON vs OFF

Exact reproduction of the 5-seed paired comparison from earlier today, with HSIKAN_TRITON_KERNEL=1 as the only change.

seed	AUC OFF	AUC ON	wall OFF (s)	wall ON (s)
0	0.9101	0.9101	1098	797
1	0.9055	0.9061	1068	790
2	0.9016	0.9028	1077	789
3	0.9093	0.9093	1080	792
4	0.9068	0.9069	1096	793
mean	0.9067 ± 0.0034	0.9070 ± 0.0029	1084	792



AUC paired delta: $+0.0004 \pm 0.0005$, $\sigma = +1.69$ — below the 2σ promotion threshold; statistically indistinguishable from kernel-OFF. The kernel-ON variance is 15% lower (0.0029 vs 0.0034).

Wall-time speedup: $1.37\times$ on average, with a remarkably tight per-seed spread of $1.35\text{--}1.38\times$. Total 5-seed wall: 90 min $\rightarrow$ 66 min, saving 24 min per such ablation.

Above SGT. 0.9070 remains $+0.010$ above the published SGT 0.897 Slashdot SOTA. The kernel preserves the position; it accelerates getting there.

6 Test coverage

33 tests organised in six tiers, all green with `HSIKAN_TRITON_KERNEL=0` (default) and `HSIKAN_TRITON_KERNEL=1` (kernel ON for live integration):

tier	coverage	status
1	forward parity vs PyTorch ref ($\leq 10^{-7}$ at 6+ shapes)	✓
2	production-layer parity vs <code>SignedKANLayer.forward</code> ($\leq 10^{-7}$)	✓
3	autograd parity ($\partial x, \partial \text{coef}, \partial W, \partial b, \leq 10^{-8}$, parametrized $d \in \{4, 16, 32\}$)	✓
4	speedup thresholds (CR $\geq 5\times$, fused inner $\geq 10\times$)	✓
5	install / uninstall idempotency	✓
6	end-to-end BA training, kernel-OFF and kernel-ON	✓

7 What is left on the table

- **Backward atomic-add contention** on hot vertex IDs is the dominant cost. Sorted-by-vertex cycle ordering would reduce contention but adds a permutation pass; deferred until profiling shows it dominates over the matmul.
- **Larger BLOCK_T at $d \geq 32$.** The $1.38\times$ `tl.dot` speedup at $d=64$ is the lowest of the three; a `BLOCK_T = 32` would better fill the tensor cores. One sensitivity sweep away.
- **Pad up at $d \in \{4, 8\}$?** Could artificially pad `BLOCK_D` to 16 and use `tl.dot` even at small d ; the loaded-but-unused W entries waste bandwidth but might still beat the loop. Untested.

8 The $d=16$ ablation: a kernel-enabled negative result

The kernel unlocked a question that was previously OOM-bound: *does wider hidden dim help on the validated SOTA recipe?*

5-seed paired comparison, identical Slashdot edge_cr recipe with `HSIKAN_TRITON_KERNEL=1` throughout, varying only the `-hidden` flag:

seed	d=4 (baseline)	d=16 (kernel-enabled)	delta
0	0.9101	0.9006	-0.0095
1	0.9055	0.8999	-0.0056
2	0.9016	0.8988	-0.0028
3	0.9093	0.8914	-0.0179
4	0.9068	0.8947	-0.0121
mean	0.9067 ± 0.0034	0.8971 ± 0.0039	$-0.0096 \pm 0.0059, \sigma = -3.66$

$d=16$ loses at every single seed. Paired $\sigma = -3.66$ — a clean, paired-validated negative result.

Interpretation. The signed-cycle aggregation acts as a strong implicit regularizer; wider hidden dim breaks it. Consistent with the Epinions h-sweep that was monotonically decreasing in h . The $d=4$ SOTA is not just a lucky regime choice — it is the architecturally correct capacity, and we now have 5-seed paired evidence to support that claim.

The kernel was the enabler. The $d=16$ run took 792 s per seed — identical to the kernel-OFF wall time at $d=4$. The `tl.dot` optimization at $d \geq 16$ absorbed the d^2 gate matvec scaling so cleanly that compute time held flat while capacity quadrupled. Without the kernel, this experiment was 3 GiB peak — impossible on the development GPU.

9 Reproducibility

```
# Run the test suite (default OFF, then forced ON)      both must pass.
python -m pytest signedkan_wip/tests/test_triton_kernels.py -v
HSIKAN_TRITON_KERNEL=1 python -m pytest signedkan_wip/tests/test_triton_kernels.py -v

# Reproduce the kernel-OFF Slashdot 5-seed (~90 min)
bash signedkan_wip/experiments/run_slashdot_edge_cr_5seed_2026_05_09.sh

# Reproduce the kernel-ON Slashdot 5-seed (~66 min, this paper)
bash signedkan_wip/experiments/run_slashdot_edge_cr_kernel_on_2026_05_09.sh

# Sensitivity sweep, parameter landscape (~2 min)
python -m signedkan_wip.src.triton_kernels_sensitivity
```

Companion documents. Full math derivation: docs/triton_kernel_integration_tutorial_2026_05_09.md and reports/triton_kernel_integration_tutorial_2026_05_09.pdf. Memory deep-dive: reports/triton_k_tldot_optimization_detail: reports/triton_kernel_tldot_report_2026_05_09.pdf.